

UNIT - I(Introduction)

1.0 Definition:

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

1.1 History of Python

- Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.
- Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, Unix shell, and other scripting languages.
- At the time when he began implementing Python, Guido van Rossum was also reading the published scripts from **Monty Python's Flying Circus**. It occurred to him that he needed a name that was short, unique, and slightly mysterious, so he decided to call the language Python.
- Python 1.0 was released on **20 February, 1991**.
- Python 2.0 was released on **16 October 2000** and had many major new features, including a cycle detecting garbage collector and support for Unicode.
Python 3.0 (which early in its development was commonly referred to as Python 3000 or py3k), a major, backwards-incompatible release, was released on **3 December 2008** after a long period of testing.
- Python is now maintained by a core development team at the institute, although **Guido van Rossum** still holds a vital role in directing its progress.

1.2 Python Features:

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same

interface on all platforms.

- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

1.3 Python Applications

Python is known for its general-purpose nature that makes it applicable in almost every domain of software development. It is the fastest-growing programming language and can develop any application.

1) Web Applications

We can use Python to develop web applications. It provides libraries to handle internet protocols such as HTML and XML, JSON, Email processing, request, BeautifulSoup, Feedparser, etc. One of Python web-framework named Django is used on Instagram. Python provides many useful frameworks, and these are given below:

- Django and Pyramid framework(Use for heavy applications)
- Flask and Bottle (Micro-framework)
- Plone and Django CMS (Advance Content management)

2) Desktop GUI Applications

The GUI stands for the Graphical User Interface, which provides a smooth interaction to any application. Python provides a Tk GUI library to develop a user interface. Some popular GUI libraries are given below.

- Tkinter or Tk
- wxWidgetM
- Kivy (used for writing multitouch applications)
- PyQt or Pyside

3) Software Development

Python is useful for the software development process. It works as a support language and can be used to build control and management, testing, etc.

- SCons is used to build control.
- Buildbot and Apache Gumps are used for automated continuous compilation and testing.
- Round or Trac for bug tracking and project management.

4) Scientific and Numeric: It consists of many scientific and mathematical libraries, which makes easy to solve complex calculations. Implementing machine learning algorithms require complex mathematical calculation. Python has many libraries for scientific and numeric such as Numpy, Pandas, Scipy, Scikit-learn, etc.

5) Business Applications

Business Applications differ from standard applications. E-commerce and ERP are an example of a business application. This kind of application requires extensively, scalability and readability, and Python provides all these features.

6) Audio or Video-based Applications

Python is flexible to perform multiple tasks and can be used to create multimedia applications. Some multimedia applications which are made by using Python are TimPlayer, cplay, etc. The few multimedia libraries are given below.

- Gstreamer
- Pyglet
- QT Phonon

7) Enterprise Applications

Python can be used to create applications that can be used within an Enterprise or an Organization. Some real-time applications are OpenERP, Tryton, Picalo, etc.

8) Image Processing Application

Python contains many libraries that are used to work with the image. The image can be manipulated according to our requirements. Some libraries of image processing are given below.

- OpenCV
- Pillow
- SimpleITK

1.4.EXPLAIN IDLE:

IDLE stands for Integrated Development and Learning Environment. IDLE is a very simple and sophisticated IDE developed primarily for beginners, and because of its simplicity, it is highly considered and recommended for educational purposes.

Some of the key features it offers are:

- Python shell with syntax highlighting.
- Multi-window text editor.
- Code autocompilation.
- Intelligent indenting.
- Program animation and stepping which allows one line of code to run at a time helpful for debugging.

How to Install IDLE?

- One way to install IDLE is to install Python from the official Python website.

You can download the latest Python versions from this website for different operating systems like windows, Linux, and mac. It also provides you the docker images for Python, which you can directly use if you have a docker installed on your system.

- Let's install Python for the Windows operating system.

Installing Python on Windows OS

- Since Python2 development has been stopped, let's install the latest Python version, i.e., 3.10.5.
- Install the Windows x86 executable installer file depending on whether you have a 32-bit or 64-bit OS and run that file. Once you run the file, a window will open, Make sure you select the install launcher for all users and add Python3.8 to Path along with the recommended installation.
- Once the installation is complete, go to the start menu and type idle and click on the same. You should now be seeing the IDLE software on your system.

Exploring IDLE Software Features

- Remember that it is not advisable to write multiple lines of code that have functions/classes in the IDLE shell. In such cases, you can go to the File option of IDLE and click on New File.
- IDLE can be customized using the options present in the Format, Edit, and Options menu.

1.5. Running Python:

- Goto **File** menu click on New File (CTRL+N) and write the code and save add.py

```
a=input(int("Enter a value "))
b=input(int("En b val"))
c=a+b
print( "The sum is",c)
```

```
OUTPUT:Enter a value 12
        En b val 12
        The sum is 24.
```

And run the program by pressing F5 or Run Module

1.6. Identifier is a name used to identify a variable, function, class, module, etc. The identifier is a combination of character digits and underscore. The identifier should start with a character or Underscore then use a digit. The characters are A-Z or a-z, an Underscore (_) , and digit (0-9). we should not use special characters (#, @, \$, %, !) in identifiers.

- **Examples of valid identifiers:**

- *var1*
- *_var1*
- *_1_var*
- *var_1*

- **Examples of invalid identifiers**

- *!var1*
- *1var*
- *1_var*
- *var#1*

Variables:

- ✓ Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.
- ✓ Based on the data type of a variable, the interpreter allocates memory and decides what can be stored in the reserved memory.
- ✓ Therefore, by assigning different data types to variables, you can store integers, decimals or characters in these variables.

Assigning Values to Variables

Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable. The equal sign (=) is used to assign values to variables.

The operand to the left of the = operator is the name of the variable and the operand to the right of the = operator is the value stored in the variable. For example –

Example: A=10

B=20

C=A+B

Print('The sum is',C)

O/P:The sum is 55.

Multiple Assignments to variables:

Python allows you to assign a single value to several variables simultaneously.

For example –

a = b = c = 1

Here, an integer object is created with the value 1, and all three variables are assigned to the same memory location. You can also assign multiple objects to multiple variables.

For example –

a, b, c = 1, 2.5, "Sree"

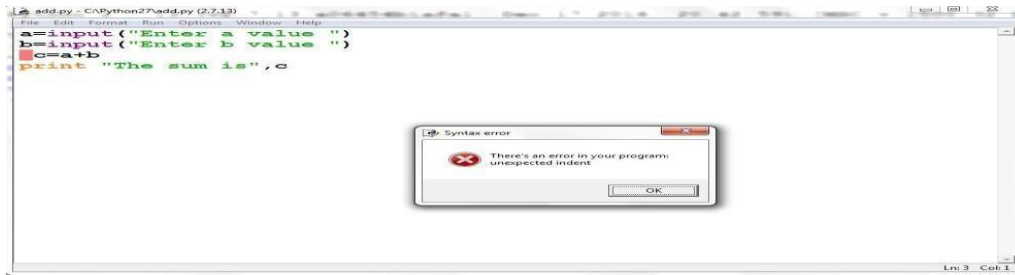
KEYWORDS

The following list shows the Python keywords. These are reserved words and you cannot use them as constant or variable or any other identifier names. All python keywords are case sensitive.

and	elif	if	print
as	else	import	raise
assert	except	in	return
break	exec	is	try
class	finally	lambda	while
continue	for	not	with
def	from	or	yield
del	global	pass	

Indentation

- Code blocks are identified by indentation rather than using symbols like curly braces. Without extra symbols, programs are easier to read.
Python does not support braces to indicate blocks of code for class and function definitions or flow control.
- Blocks of code are denoted by line indentation. All the continuous lines indented with same number of spaces would form a block.
- Python strictly follow indentation rules to indicate the blocks.



1.7. Standard Data Types:

The data stored in memory can be of many types. For example, a person's age is stored as a numeric value and his or her address is stored as alphanumeric characters. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.

Python has five standard data types:

- Numbers
- Boolean
- Tuple
- String
- List
- Dictionary

Python Numbers:

Number data types store numeric values. Number objects are created when you assign a value to them.

Python supports four different numerical types:

- int (signed integers)
- long (long integers, they can also be represented in octal and hexadecimal)
- float (floating point real values)
- complex (complex numbers)

Python allows you to use a lowercase L with long, but it is recommended that you use only an uppercase L to avoid confusion with the number 1. Python displays long integers with an uppercase L.

A complex number consists of an ordered pair of real floating-point numbers denoted by $x + yj$, where x is the real part and b is the imaginary part of the complex number.

For example:

Program:

```
a = 3
b = 2.65
c = 98657412345L
d = 2+5j
print('a value is=',a)
print('datatype of a=',type(a))
print('b value is=',b)
print('datatype of d=',type(d))
print('c value is=',c)
```

Output:

```
a value is =3
datatype of a =<class 'int'>
b value is =2.65
datatype of d=<class 'complex'>
c value is = 98657412345L
```

Python Strings:

Strings in Python are identified as a contiguous set of characters represented in the

quotation marks. Python allows for either pairs of single or double quotes. Subsets of strings can be taken using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the string and working their way from -1 at the end.

The plus (+) sign is the string concatenation operator and the asterisk (*) is the repetition operator. For example:

Program:

```
str="WELCOME"
print( str) # Prints complete string
print( str[0]) # Prints first character of the string
print (str + "CSE" )# Prints concatenated string
```

Output: WELCOME

W

WELCOMECSE

Python Boolean:

Booleans are identified by True or False.

Example a =True

b = False

print(a)

print (b)

Output:

True

False

List

Lists are just like the arrays, declared in other languages which is a ordered collection of data. It is very flexible as the items in a list do not need to be of the same type.

Creating List

Lists in Python can be created by just placing the sequence inside the square brackets[].

List = ["sree", "sita", "ram"]

print("Accessing element from the list")

print(List[0])

print(List[2])

Output:Accessing element from
the list

Sree

ram

1.8. Declaring a variable:

Python has no command for declaring a variable. A variable is created when some value is assigned to it. The value assigned to a variable determines the data type of that variable.

Thus, declaring a variable in Python is very simple.

- Just name the variable
- Assign the required value to it
- The data type of the variable will be automatically determined from the value assigned, we need not define it explicitly.

Declare an integer variable

To declare an integer variable -

- Name the variable
- Assign an integer value to it

Example

x=2

print(x)

Output

2

<class'int'>

```
print(type(x))
```

This is how you declare a integer variable in Python. Just name the variable and assign the required value to it. The datatype is automatically determined.

Declare a string variable

Assign a string value to the variable and it will become a string variable. In Python, the string value can be assigned in single quotes or double quotes.

Example

```
x='2'  
print(x)  
print(type(x))
```

Output

```
2  
<class 'str'>
```

Declare a float variable

A float variable can be declared by assigning float value. Another way is by typecasting.

We will use both.

Example

```
x=2.0  
print(x)  
print(type(x))  
y=float(2)  
print(y)  
print(type(y))
```

Output

```
2.0  
<class 'float'>  
2.0  
<class 'float'>
```

Unlike some other languages, where we can assign only the value of the defined data type to a variable. This means an integer variable can only be assigned an integer value throughout the program. But, in Python, the variables are not of a particular datatype. Their datatype can be changed even after it is set.

The following example will clarify the above concept.

Example

```
x=10  
print(x)  
print(type(x))  
x="abc"  
print(x)  
print(type(x))
```

Output

```
10  
<class 'int'>  
abc  
<class 'str'>
```

The variable x was of type int. Later when string value is assigned to it, it changes into a string variable.

1.9. INPUT Function:

To get input from the user you can use the input function. When the input function is called the program stops running the program, prompts the user to enter something at the keyboard by printing a string called the prompt to the screen, and then waits for the user to press the Enter key. The user types a string of characters and presses enter. Then the input

function returns that string and Python continues running the program by executing the next statement after the input statement.

Python provides the function `input()`. `input` has an optional parameter, which is the prompt string.

For example

Name=`input("enter your name")`

Print(`"the name is ",Name`)

enter your name "sree"
the name is sree

OUTPUT Function:

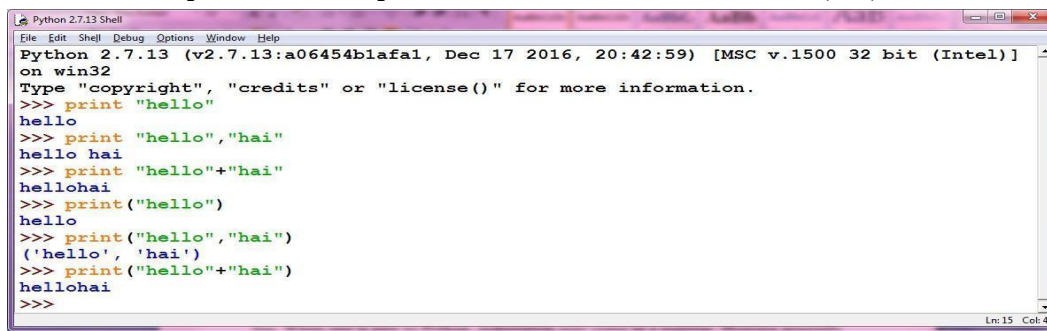
We use the `print()` function or `print` keyword to output data to the standard output device (screen). This function prints the object/string written in function.

The actual syntax of the `print()` function is

`print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)`

Here, objects is the value(s) to be printed.

The `sep` separator is used between the values. It defaults into a space character. After all values are printed, `end` is printed. It defaults into a new line (`\n`).



```
Python 2.7.13 Shell
File Edit Shell Debug Options Window Help
Python 2.7.13 (v2.7.13:a06454blafal, Dec 17 2016, 20:42:59) [MSC v.1500 32 bit (Intel)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>> print "hello"
hello
>>> print "hello", "hai"
hello hai
>>> print "hello"+"hai"
hellohai
>>> print("hello")
hello
>>> print("hello", "hai")
('hello', 'hai')
>>> print("hello"+"hai")
hellohai
>>>
```

1.10. Input Output format:

`input()` always returns a string. If you want a numeric type, then you need to convert the string to the appropriate type with the built-in `int()`, `float()`, or `complex()` function:

```
>>> number = input("Enter a number: ")
```

Enter a number: 50

```
>>> print(number + 100)
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: must be str, not int

```
>>> number = int(input("Enter a number: "))
```

Enter a number: 50

```
>>> print(number + 100)
```

150

Output Formatting using String modulo operator (%):

The String formatting can also be done by using % operator. It interprets the left argument like printf() format string which is applied to right argument. In python, we don't have printf() function. For this purpose, modulo operator is overloaded by string class in order to perform string formatting. Hence it is often known as String modulo operator.

#integer and float value

```
print("integer : % 2d, float : % 5.2f" %(1, 5.333))
```

Output:

integer : 1, float : 5.33

Output Formatting using Format method:

Formatting output using format method requires more manual effort. User uses {} to mark the substitution of variable and provides detailed formatting directives, but user should provide the information for formatting.

EX

using format() method

```
print('{} and {}'.format('Python', 'Java'))
```

a position of the object

```
print('{0} is {1}'.format('Python', 'Programming Language'))
```

```
print('{1} is {0}'.format('Python', 'Programming Language'))
```

Output:

Python and Java

Python is ProgrammingLanguage

Programming Language is Python

Output Formatting using String Method:

We can format the output by using String method by String slicing and concatenations. Through these methods, output can be formatted in exclusive way. Some of the methods are str.rjust(), str.center() and str.ljust().

ex

```
stmt = "Output Formatting Functions"
```

Printing the right aligned sentence

```
print ("Right aligned statement: ")
```

```
print (stmt.rjust(50, '*'))
```

Output:

Right aligned statement:

```
*****Output Formatting
Functions
```

1.11. usage of comments: Using comments in programs makes our code more understandable. It makes the program more readable which helps us remember why certain blocks of code were written.

Other than that, comments can also be used to ignore some code while testing other blocks of code. This offers a simple way to prevent the execution of some lines or write a quick pseudo-code for the program.

1.12. Types of Operators:

Python language supports the following types of operators.

- Arithmetic Operators +, -, *, /, %, **, //
- Comparison (Relational) Operators =, !=, <, >, <=, >=
- Assignment Operators =, +=, -=, *=, /=, %=, **=, //=
- Logical Operators and, or, not
- Bitwise Operators &, |, ^, ~, <<, >>
- Membership Operators in, not in
- Identity Operators is, is not

Arithmetic Operators:

Some basic arithmetic operators are +, -, *, /, %, **, and //. You can apply these operators on numbers as well as variables to perform corresponding operations.

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	a + b = 30
- Subtraction	Subtracts right hand operand from left hand operand.	a - b = -10
* Multiplication	Multiplies values on either side of the operator	a * b = 200
/ Division	Divides left hand operand by right hand operand	b / a = 2
% Modulus	Divides left hand operand by right hand operand and returns remainder	b % a = 0
** Exponent	Performs exponential (power) calculation on operators	a**b = 10 to the power 20
// Floor Division	The division of operands where the result is the quotient in which the digits after the decimal point are removed.	9//2 = 4 and 9.0//2.0 = 4.0

Comparison (Relational) Operators

These operators compare the values on either sides of them and decide the relation among them. They are also called Relational operators.

Operator	Description	Example
= =	If the values of two operands are equal, then the condition becomes true.	(a == b) is not true.
!=	If values of two operands are not equal, then condition becomes true.	(a != b) is true.
< >	If values of two operands are not equal, then condition becomes true.	(a <> b) is true. This is similar to != operator.
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	(a > b) is not true.
<	If the value of left operand is less than the value of right operand, then condition becomes true.	(a < b) is true.
> =	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	(a >= b) is not true.
< =	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	(a <= b) is true.

Assignment Operators

Operator	Description	Example
----------	-------------	---------

=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c - a$
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	$c *= a$ is equivalent to $c = c * a$
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	$c /= a$ is equivalent to $c = c / a$
%= Modulus AND	It takes modulus using two operands and assign the result to left operand	$c \% a$ is equivalent to $c = c \% a$
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	$c ** a$ is equivalent to $c = c ** a$
//= Floor Division	It performs floor division on operators and assign value to the left operand	$c //= a$ is equivalent to $c = c // a$

Logical Operators

Operator	Description	Example
And Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
Or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not (a and b) is false.

Bitwise Operators

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands.	$(a \& b) = 12$ (means 0000 1100)
 Binary OR	It copies a bit if it exists in either operand.	$(a b) = 61$ (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	$(a \wedge b) = 49$ (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	$(\sim a) = -61$ (means 1100 0011 in 2's complement form due to a signed binary number).

<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

Membership Operators

Python's membership operators test for membership in a sequence, such as strings, lists, or tuples.

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not find a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Identity Operators

Identity operators compare the memory locations of two objects.

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

1.13 Boolean values

- Booleans represent one of two values: True or False.

In programming you often need to know if an expression is True or False.

- You can evaluate any expression in Python, and get one of two answers, True or False.
- When you compare two values, the expression is evaluated and Python returns the Boolean answer:

Example

```
print(10 > 9)
print(10 == 9)
print(10 < 9)
```

When you run a condition in an if statement, Python returns True or False:

Example**Print a message based on whether the condition is True or False:**

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

1.14. Explain Operator Rules.**Python Operators Precedence**

The following table lists all operators from highest precedence to lowest.

Operator	Description
()	Parenthesis
**	Exponentiation (raise to the power)
~ x, +x, -x	Complement, unary plus and minus
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

1.15. State the purpose of modules.

- We use modules to break down large programs into small manageable and organized files.
- Furthermore, modules provide reusability of code.
- We can define our most used functions in a module and import it, instead of copying their definitions into different programs.

1.16. Define functions.

A function is a block of organized, reusable code that is used to perform a single, related action.

- Once a function is written, it can be reused as and when required. So, functions are also called reusable code.
- Functions provide modularity for programming. A module represents a part of the program. Usually, a programmer divides the main task into smaller sub tasks called modules.
- Code maintenance will become easy because of functions. When a new feature has to be

added to the existing software, a new function can be written and integrated into the software.

- When there is an error in the software, the corresponding function can be modified without disturbing the other functions in the software.

Syntax: def function_name(parameters):

 """docstring"""

 # body of the function

 return expression

1.17. List types of functions (Built-in, User defined)

- The use of functions in a program will reduce the length of the program.
Python gives you many built-in functions like `sqrt()`, etc. but you can also create your own functions. These functions are called user-defined functions.

1.18 . Built-in Functions:

Built-in functions are the functions that are already written or defined in python. We only need to remember the names of built-in functions and the parameters used in the functions. As these functions are already defined so we do not need to define these functions. Some built-in functions of Python.

Function name	Description
<code>len()</code>	It returns the length of an object/value.
<code>list()</code>	It returns a list.
<code>max()</code>	It is used to return maximum value from a sequence (list,sets) etc.
<code>min()</code>	It is used to return minimum value from a sequence (list,sets) etc.
<code>open()</code>	It is used to open a file.
<code>print()</code>	It is used to print statement.
<code>str()</code>	It is used to return string object/value.
<code>sum()</code>	It is used to sum the values inside sequence.
<code>type()</code>	It is used to return the type of object.
<code>tuple()</code>	It is used to return a tuple.

Ex: `x=[1,2,3,4,5]`

`print(len(x))`

`print(type(x))`

Output:

5

<class'int'>

2). User-Defined Functions:

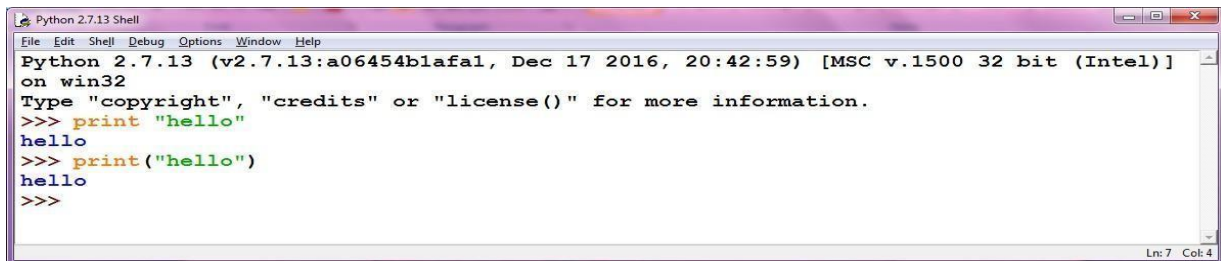
The functions defined by a programmer to reduce the complexity of big problems and to use that function according to their need. This type of functions is called **user-defined functions**.

EX:	OUTPUT:
X=3	7
Y=4	
def add():	
print(x+y)	
add()	

1.19. Give the Steps in develop a simple python program and execution:.

a. Running Python Interpreter:

Python comes with an interactive interpreter. When you type python in your shell or command prompt, the python interpreter becomes active with a >>> prompt and waits for your commands. Now you can type any valid python expression at the prompt. Python reads the typed expression, evaluates it and prints the result.



b. Running Python Scripts in IDLE:

- Goto **File** menu click on New File (CTRL+N) and write the code and save add.py

```
a=input(int("Enter a value "))
b=input(int("En b val"))
c=a+b
print( "The sum is",c)
```

OUTPUT: Enter a value 12
En b val 12
The sum is 24.

- And run the program by pressing F5 or Run Module.

C. Running python scripts in Command Prompt:

- Before going to run we have to check the PATH in environment variables.
- Open your text editor, type the following text and save it as hello.py.

```
print ("hello")
```
- And run this program by calling python hello.py. Make sure you change to the directory where you saved the file before doing it.

